



Manual de Instalación

Sistema de Gestión de Inventario
«Librería El Saber»

Aprendices

Carlos Iván Perdomo Perdomo

Luz Darlys González Torres

Instructor

Ingeniero Jairo Antonio Muñoz Arango

Programa de formación

Tecnólogo en Análisis y Desarrollo de Software (ADSO) — Ficha 3118323

Centro de formación

Centro Agropecuario La Granja — SENA Regional Tolima

Control del documento

Versión	Fecha	Descripción del cambio	Elaborado por
1.0	Agosto de 2026	Versión inicial para la sustentación del proyecto final	Carlos I. Perdomo · Luz D. González

Contenido

1. Introducción y alcance

2. Características del software

3. Servicios que componen el sistema

4. Requisitos de hardware y software

5. Paso 0 — Instalar los prerequisites

6. Paso 1 — Obtener el código fuente

7. Paso 2 — Crear la base de datos

8. Paso 3 — Configurar y arrancar el backend
9. Paso 4 — Configurar y arrancar el frontend

10. Paso 5 — Verificación funcional (checklist)

11. Ejecutar las pruebas automatizadas

12. Ejecución en modo producción local

13. Despliegue en la nube (Railway)

14. Solución de problemas de instalación

15. Anexo — Empaquetado para la entrega

1. Introducción y alcance

Este manual explica **cómo habilitar y ejecutar el proyecto desde cero** en un computador sin nada preinstalado, hasta dejar el sistema funcionando y verificado. Cubre: instalación de prerequisites, creación de la base de datos, configuración de variables de entorno, arranque del backend y del frontend, verificación funcional, ejecución de pruebas y el despliegue en la nube (Railway).

Convención: los comandos se muestran en bloques oscuros. Se ejecutan en la terminal de Windows (PowerShell o CMD) salvo que se indique lo contrario. La carpeta raíz del proyecto se llama `proyecto-inventario` (en GitHub: `SGI-Libreria-el-Saber`).

2. Características del software

Característica	Detalle
Nombre	SGI — Sistema de Gestión de Inventario «Librería El Saber»
Tipo de aplicación	Aplicación web SPA (Single Page Application) cliente–servidor
Frontend	React 19 + Vite 7 + Bootstrap 5 (carpeta <code>cliente/</code>)
Backend	Node.js 20+ con Express 5 — API REST (carpeta <code>servidor/</code>)
Base de datos	MySQL 8 — esquema <code>inventario_libreria</code> , 10 tablas con prefijo <code>mdc_</code> , motor InnoDB, charset utf8mb4
Autenticación	JWT (HS256, expiración 8 h) + contraseñas con hash bcrypt
Autorización	RBAC con dos roles: Administrador (1) y Vendedor (2)
Almacenamiento de imágenes	Cloudinary (nube) o disco local <code>servidor/uploads/portadas/</code> si no se configura Cloudinary
Repositorio	<code>https://github.com/cperdomope/SGI-Libreria-el-Saber</code> (rama <code>main</code>)
Licencia / uso	Proyecto académico — SENA ADSO, Ficha 3118323

3. Servicios que componen el sistema

Para que el sistema funcione correctamente deben estar activos estos servicios:

Servicio	Puerto local	Función	¿Obligatorio?
MySQL 8	3306	Almacena todos los datos (libros, ventas, usuarios...)	Sí
Backend Express (API REST)	3000	Lógica de negocio, seguridad y acceso a la BD	Sí
Frontend Vite (React)	5173	Interfaz que usa el usuario en el navegador	Sí
Cloudinary (servicio externo)	—	Guarda las portadas de los libros en la nube	No — sin él las imágenes se guardan en disco local

El orden de arranque recomendado es: **1) MySQL → 2) Backend → 3) Frontend**.

4. Requisitos de hardware y software

4.1 Hardware mínimo recomendado

- Procesador de 2 núcleos o superior.
- 4 GB de RAM (8 GB recomendado para desarrollar con comodidad).
- 2 GB de espacio libre en disco (proyecto + dependencias + MySQL).

4.2 Software requerido

Programa	Versión mínima	¿Para qué?	Descarga
Node.js (incluye npm)	20.x	Ejecutar el backend y compilar el frontend	nodejs.org
MySQL Server	8.x	Base de datos	dev.mysql.com/downloads
Git	2.x	Clonar el repositorio (opcional si se usa el ZIP)	git-scm.com
Navegador	Chrome / Edge / Firefox actual	Usar la aplicación	—
MySQL Workbench o HeidiSQL	—	Opcional: administrar la BD gráficamente	—

5. Paso 0 — Instalar los prerequisites

5.1 Node.js

1. Descargue el instalador LTS (versión 20 o superior) desde <https://nodejs.org>.
2. Ejecute el instalador con las opciones por defecto (incluye npm).
3. Verifique en una terminal nueva:

```
node -v    # debe mostrar v20.x o superior
npm -v     # debe mostrar 10.x o superior
```

5.2 MySQL 8

1. Descargue «MySQL Installer» desde <https://dev.mysql.com/downloads/installer/>.
2. Instale **MySQL Server 8** (puede incluir Workbench). Durante la instalación defina la contraseña del usuario `root` — **anótela**, se usa en el Paso 2 y en el archivo `.env`.
3. Verifique que el servicio esté activo:

```
mysql --version
# y para probar la conexión (pedirá la contraseña de root):
mysql -u root -p -e "SELECT VERSION();" 
```

5.3 Git (opcional)

```
git --version    # git version 2.x
```

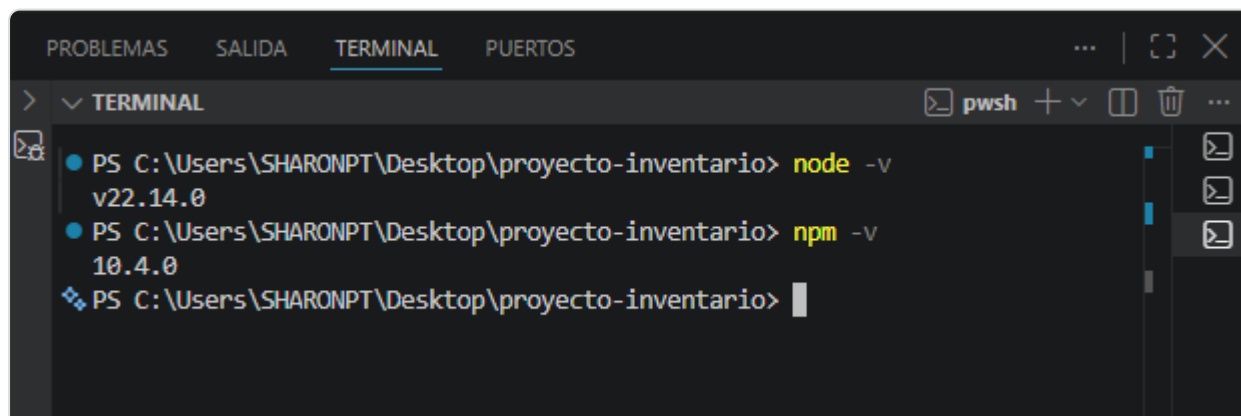


FIGURA 1. Verificación de los prerequisites instalados.

6. Paso 1 — Obtener el código fuente

Opción A — Clonar con Git (recomendada):

```
git clone https://github.com/cperdomope/SGI-Libreria-el-Saber.git
cd SGI-Libreria-el-Saber
```

Opción B — Desde el archivo comprimido de la entrega: descomprima el `.zip` / `.rar` del proyecto en una carpeta sin espacios ni tildes en la ruta (por ejemplo `C:\proyectos\proyecto-inventario`) y abra una terminal en esa carpeta.

La estructura que debe verse es:

```
proyecto-inventario/
├─ base_datos/sgi_libreria_completo.sql  # script completo de la BD
├─ cliente/                             # frontend React + Vite
├─ servidor/                            # backend Express (API REST)
├─ manuales/                            # manuales del proyecto (este documento)
└─ README.md
```

7. Paso 2 — Crear la base de datos

El script `base_datos/sgi_libreria_completo.sql` hace todo el trabajo: crea la base `inventario_libreria` con charset `utf8mb4`, las **10 tablas** con sus llaves foráneas e índices, **3 vistas** de consulta y los **datos semilla** (2 roles, 3 usuarios, 7 categorías, 8 autores, 8 libros, 7 clientes y 4 proveedores).

Desde la carpeta raíz del proyecto ejecute (pedirá la contraseña de root):

```
mysql -u root -p < base_datos/sgi_libreria_completo.sql
```

Alternativa gráfica: abra el archivo en **MySQL Workbench** y ejecútelo completo (ícono del rayo).

Advertencia: el script inicia con `DROP DATABASE IF EXISTS inventario_libreria;` — si la base ya existía, la borra y la crea limpia. Ejecútelo solo en instalaciones nuevas o cuando quiera reiniciar los datos.

Verificación rápida:

```
mysql -u root -p -e "USE inventario_libreria; SHOW TABLES;"
# Debe listar: mdc_autores, mdc_categorias, mdc_clientes, mdc_detalle_ventas,
# mdc_libros, mdc_movimientos, mdc_proveedores, mdc_roles, mdc_usuarios, mdc_ventas
```

8. Paso 3 — Configurar y arrancar el backend

8.1 Crear el archivo de variables de entorno

```
cd servidor
copy .env.example .env    # en PowerShell también sirve: cp .env.example .env
```

Abra `servidor/.env` con el Bloc de notas o VS Code y complete los valores:

Variable	Valor de ejemplo	Descripción
PORT	3000	Puerto donde escucha la API
DB_HOST	localhost	Servidor de MySQL
DB_PORT	3306	Puerto de MySQL
DB_USER	root	Usuario de MySQL
DB_PASSWORD	(su contraseña de root)	Contraseña definida al instalar MySQL
DB_NAME	inventario_libreria	Nombre de la base creada en el Paso 2
DB_SSL	false	false en local; true con BD en la nube
JWT_SECRET	(cadena larga aleatoria)	Clave con la que se firman los tokens. Sin ella el con: node -e "console.log(require('crypto').randomBy
JWT_EXPIRY	8h	Duración de la sesión
NODE_ENV	development	development en local, production en la nu
CORS_ORIGIN	http://localhost:5173	Origen del frontend autorizado
CLOUDINARY_CLOUD_NAME CLOUDINARY_API_KEY CLOUDINARY_API_SECRET	(opcionales)	Credenciales de cloudinary.com para guardar p vacías, las imágenes se guardan en servidor/

Nunca suba el archivo `.env` a GitHub: contiene secretos. El repositorio ya lo excluye mediante `.gitignore`.

8.2 Instalar las dependencias

```
npm install    # descarga Express, mysql2, jsonwebtoken, bcryptjs, helmet, etc.
```

8.3 Generar las contraseñas reales de los usuarios de prueba

El script SQL deja los tres usuarios con un `password_hash` **provisional (placeholder)**, por lo que todavía **no es posible iniciar sesión**. Hay que generar los hashes bcrypt reales y actualizarlos en la base de datos:

Nota: algunos comentarios del script SQL mencionan `node scripts/reset_password.js`; ese script se eliminó del repositorio por seguridad (contenía contraseñas en texto plano). El procedimiento vigente es el siguiente.

a) Genere los tres hashes (desde la carpeta `servidor/`):

```
node -e "const b=require('bcryptjs'); ['Luzd12345','cip123','vendedor123'].forEach(p => console.log(b.hashSync(p)))"
```

b) Copie cada hash impreso (empiezan por `$2b$10$...`) y ejecútelos en MySQL:

```
mysql -u root -p inventario_libreria

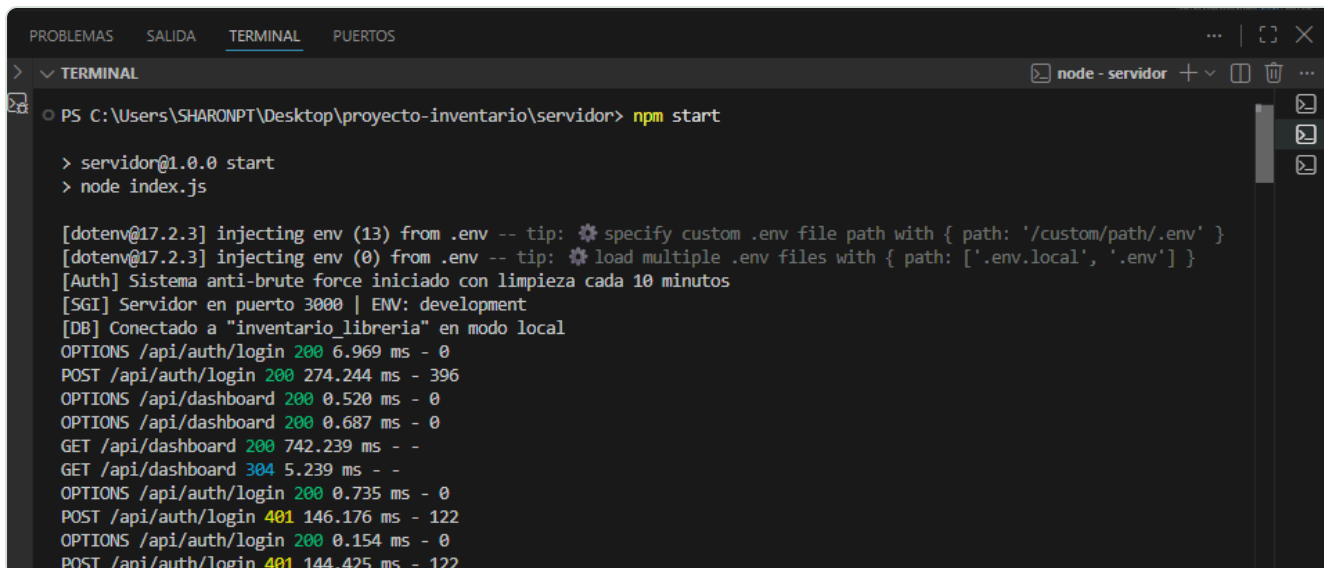
UPDATE mdc_usuarios SET password_hash = '<hash_de_Luzd12345>' WHERE email = 'ldarlys@sena.edu.cu';
UPDATE mdc_usuarios SET password_hash = '<hash_de_cip123>' WHERE email = 'cip@sena.edu.cu';
UPDATE mdc_usuarios SET password_hash = '<hash_de_vendedor123>' WHERE email = 'michelle@sena.edu.cu';
EXIT;
```

8.4 Arrancar el servidor

```
npm start
# Salida esperada:
# [SGI] Servidor en puerto 3000 | ENV: development
# [DB] Conectado a "inventario_libreria" en modo local
```

Verifique el **health check** abriendo `http://localhost:3000` en el navegador. Debe responder un JSON como este (comprueba servidor y conexión a la BD):

```
{ "estado": "OK", "bd": "conectada", "sistema": "SGI Librería El Saber", ... }
```

```
PROBLEMAS  SALIDA  TERMINAL  PUERTOS
> v TERMINAL
node - servidor + - - -
PS C:\Users\SHARONPT\Desktop\proyecto-inventario\servidor> npm start

> servidor@1.0.0 start
> node index.js

[dotenv@17.2.3] injecting env (13) from .env -- tip: ⚙ specify custom .env file path with { path: '/custom/path/.env' }
[dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙ load multiple .env files with { path: ['.env.local', '.env'] }
[Auth] Sistema anti-brute force iniciado con limpieza cada 10 minutos
[SGI] Servidor en puerto 3000 | ENV: development
[DB] Conectado a "inventario_libreria" en modo local
OPTIONS /api/auth/login 200 6.969 ms - 0
POST /api/auth/login 200 274.244 ms - 396
OPTIONS /api/dashboard 200 0.520 ms - 0
OPTIONS /api/dashboard 200 0.687 ms - 0
GET /api/dashboard 200 742.239 ms - -
GET /api/dashboard 304 5.239 ms - -
OPTIONS /api/auth/login 200 0.735 ms - 0
POST /api/auth/login 401 146.176 ms - 122
OPTIONS /api/auth/login 200 0.154 ms - 0
POST /api/auth/login 401 144.425 ms - 122
```

FIGURA 2. Backend en ejecución y health check exitoso.

Modo desarrollo con recarga automática: `npm run dev` (usa nodemon: reinicia el servidor al guardar cambios en el código).

9. Paso 4 — Configurar y arrancar el frontend

9.1 Variables de entorno del cliente

En una **terminal nueva** (deje el backend corriendo en la anterior):

```
cd cliente
copy .env.example .env
```

El archivo `cliente/.env` solo necesita una línea, que indica dónde está la API:

```
VITE_API_URL=http://localhost:3000/api
```

9.2 Instalar dependencias y arrancar

```
npm install      # descarga React, Bootstrap, Axios, Recharts, jsPDF, xlsx...
npm run dev
# Salida esperada:
#   VITE v7.x  ready in ____ ms
#   → Local:   http://localhost:5173/
```

Abra `http://localhost:5173` en el navegador: debe aparecer la pantalla de login del SGI.

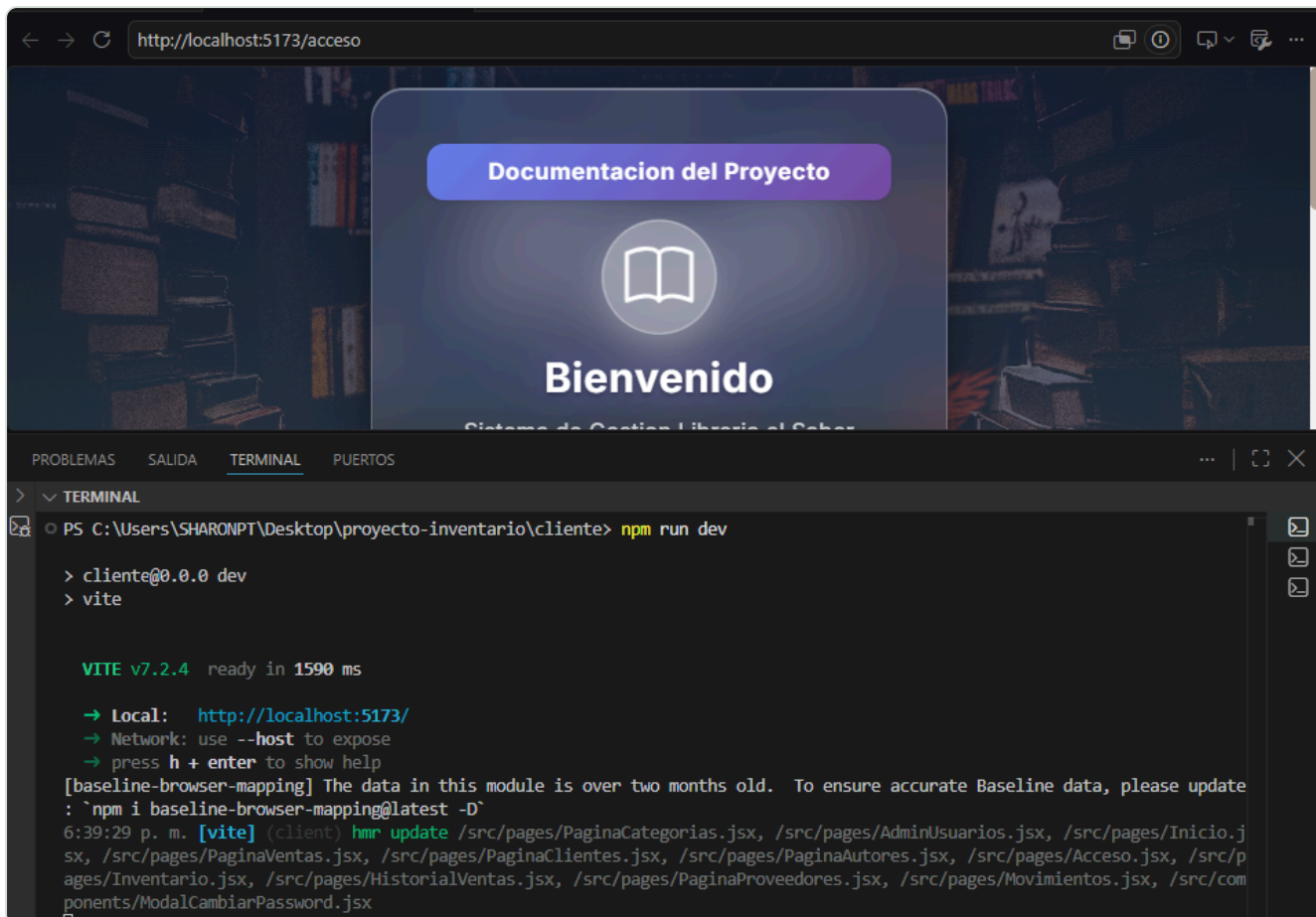


FIGURA 3. Frontend en ejecución — pantalla de acceso.

10. Paso 5 — Verificación funcional (checklist)

Con los tres servicios activos, confirme que la instalación quedó correcta:

#	Prueba	Resultado esperado
1	Abrir <code>http://localhost:3000</code>	JSON con <code>"estado": "OK"</code> y <code>"bd": "conectada"</code>
2	Abrir <code>http://localhost:5173</code>	Se ve la pantalla de login
3	Iniciar sesión con <code>ldarly@sena.edu.co / Luzd12345</code>	Entra al Dashboard con tarjetas y gráficas
4	Ir a Logística → Inventario	Se listan los 8 libros de ejemplo
5	Registrar una ENTRADA en Movimientos	El stock del libro aumenta y aparece en el Kardex
6	Registrar una venta en el POS	Mensaje de éxito y el stock disminuye
7	Descargar el ticket PDF desde el Historial	Se descarga el ticket de la venta
8	Cerrar sesión e ingresar como <code>michelle@sena.edu.co / vendedor123</code>	Entra directo al POS y NO ve Dashboard/Movimientos/Usuarios

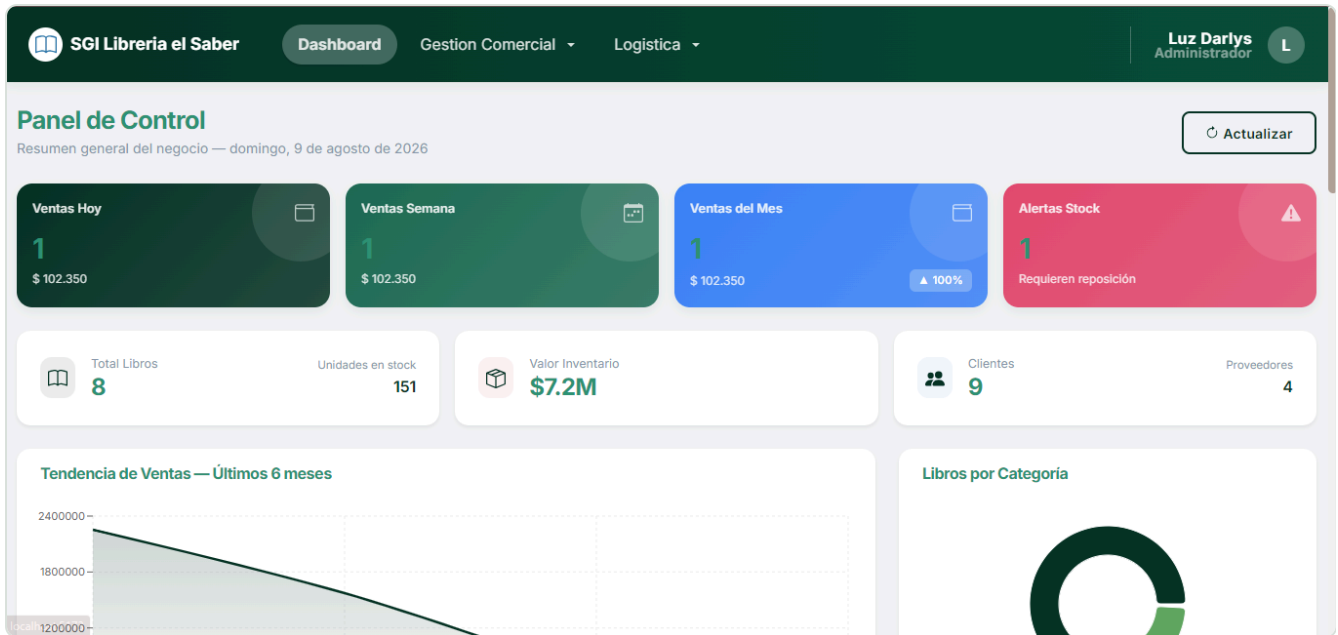


FIGURA 4. Sistema instalado y verificado.

11. Ejecutar las pruebas automatizadas

El backend incluye 8 suites de pruebas con **Jest + Supertest** que validan login y JWT, permisos por rol (RBAC), CRUDs, validaciones de negocio y el anti-fraude de totales en ventas. Requieren la base de datos activa:

```
cd servidor
npm test
```

```
> TERMINAL
● PS C:\Users\SHARONPT\Desktop\proyecto-inventario\servidor> npm test

> servidor@1.0.0 test
> jest --testPathPatterns=pruebas --forceExit --detectOpenHandles

POST /api/auth/login 401 388.729 ms - 122
POST /api/auth/login 200 361.243 ms - 415
GET /api/usuarios 401 1.428 ms - 91
GET /api/usuarios 403 13.180 ms - 117
PASS pruebas/usuarios.test.js (8.745 s)
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (12) from .env -- tip: ⚙️ load multiple .env files with { path: ['.env.local', '.env'] }

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [dotenv@17.2.3] injecting env (0) from .env -- tip: ✅ audit secrets and track compliance: https://dotenvx.com/ops

      at _log (node_modules/dotenv/lib/main.js:142:11)

    console.log
      [DB] Conectado a "inventario_libreria" en modo local

      at log (config/db.js:121:11)

PATCH /api/usuarios/cambiar-password 401 10.592 ms - 91
POST /api/auth/login 401 418.515 ms - 122
POST /api/auth/login 200 429.608 ms - 415
GET /api/movimientos 401 0.941 ms - 91
POST /api/movimientos 401 6.750 ms - 91
PASS pruebas/movimientos.test.js
  ● Console

    console.log
      [dotenv@17.2.3] injecting env (12) from .env -- tip: ⚙️ enable debug logging with { debug: true }
```

FIGURA 5. Ejecución de las pruebas automatizadas del backend.

12. Ejecución en modo producción local

12.1 Compilar el frontend

```
cd cliente
npm run build          # genera la carpeta dist/ optimizada
npx serve -s dist      # la sirve como SPA (el mismo comando usado en Railway)
```

12.2 Backend con PM2 (gestor de procesos)

PM2 mantiene el servidor vivo: lo reinicia si falla o si supera 200 MB de RAM (configuración incluida en `servidor/pm2.config.js`):

```
npm install -g pm2
cd servidor
pm2 start pm2.config.js --env production
pm2 status          # ver el estado del proceso "sgi-backend"
pm2 logs sgi-backend # ver los logs en tiempo real
pm2 stop sgi-backend # detenerlo
```

13. Despliegue en la nube (Railway)

El proyecto está desplegado en **Railway** con tres servicios conectados al repositorio de GitHub:

Servicio	URL / Host de producción	Configuración clave
Frontend	<code>https://sgi-libreria-el-saber-production.up.railway.app</code>	Build: <code>npm run build</code> · Inicio: <code>serve -s dist -l \$PORT</code> (definido en <code>cliente/railway.json</code>)
Backend	<code>https://friendly-kindness-production-06fe.up.railway.app</code>	Root Directory: <code>servidor</code> · Inicio: <code>node index.js</code> · Health check: <code>/</code> (definido en <code>servidor/railway.json</code>)
Base de datos	<code>autorack.proxv.rlwy.net:39238</code> — BD <code>inventario_libreria</code>	Plugin MySQL de Railway; inyecta automáticamente <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_NAME</code>

13.1 Pasos generales para replicar el despliegue

1. Crear el proyecto en `railway.app` y conectar el repositorio de GitHub.
2. Agregar el **plugin MySQL** y ejecutar en él el script `sgi_libreria_completo.sql` (conectándose al host público con Workbench). Actualizar los `password_hash` como en la sección 8.3.

3. Crear el servicio del **backend** con Root Directory `servidor` y configurar en su panel las variables `JWT_SECRET`, `JWT_EXPIRY`, `DB_SSL=true` y las de Cloudinary.
4. Crear el servicio del **frontend** con la variable `VITE_API_URL` apuntando a la URL pública del backend + `/api`.
5. Generar los dominios públicos de ambos servicios desde Railway.

13.2 Particularidades aprendidas en este despliegue

- **Trust proxy:** Railway pone la API detrás de un proxy inverso; el backend incluye `app.set('trust proxy', 1)` para que el rate-limiter lea la IP real desde `X-Forwarded-For`. Sin esto aparece el error `ERR_ERL_UNEXPECTED_X_FORWARDED_FOR`.
- **SPA routing:** el frontend se sirve con `serve -s dist` para que rutas como `/inventario` no den 404 al recargar.
- **Puerto:** Railway inyecta la variable `PORT` (8080); el código la usa con `process.env.PORT || 3000`.
- **CORS:** el backend acepta el dominio del frontend de producción (lista definida en `servidor/app.js` más la variable `CORS_ORIGIN`).
- **HTTPS:** lo gestiona Railway automáticamente con sus certificados.

14. Solución de problemas de instalación

Síntoma / error	Causa probable	Solución
FATAL: Variables de entorno faltantes: DB_HOST...	No existe servidor/.env o está incompleto	Crear el .env desde .env.example y completar todas las variables (sección 8.1)
JWT_SECRET no está configurado al arrancar	Falta la variable JWT_SECRET	Generarla y agregarla al .env (sección 8.1)
[DB] Error de conexión: ER_ACCESS_DENIED_ERROR	Usuario o contraseña de MySQL incorrectos	Corregir DB_USER / DB_PASSWORD en el .env
ECONNREFUSED o "bd": "desconectada" en el health check	MySQL no está corriendo	Iniciar el servicio: services.msc → «MySQL80» → Iniciar
ER_BAD_DB_ERROR: Unknown database	No se ejecutó el script SQL	Ejecutar el Paso 2
Login siempre dice «Credenciales incorrectas» con los usuarios de prueba	Los password_hash siguen siendo el placeholder del script	Generar los hashes reales y ejecutar los UPDATE (sección 8.3)
EADDRINUSE: address already in use :::3000	Otro proceso ocupa el puerto 3000	Cerrar el proceso anterior o cambiar PORT en el .env
El frontend carga pero no muestra datos / errores CORS en la consola	VITE_API_URL mal configurada o backend apagado	Revisar cliente/.env, reiniciar npm run dev (Vite solo lee el .env al arrancar) y verificar el backend
'node' no se reconoce como un comando...	Node.js no instalado o terminal abierta antes de instalarlo	Instalar Node y abrir una terminal nueva
Imágenes de portada no se ven en producción	Variables de Cloudinary sin configurar en Railway	Configurarlas en el panel del servicio backend

15. Anexo — Empaquetado para la entrega

La entrega en plataforma exige: el **proyecto comprimido** (.zip o .rar), los **manuales en PDF** y la **base de datos en .sql**. Sugerencia de preparación:

1. Exporte cada manual a PDF **desde el archivo local** (doble clic al .html de la carpeta manuales/, no desde una URL de internet): en Chrome/Edge presione **Ctrl + P** → destino «Guardar como PDF» → «Más ajustes» → márgenes «Predeterminados» → **active «Gráficos de fondo»** y **desactive «Encabezados y pies de página»** — esta última casilla es la que evita que el navegador imprima automáticamente el título, la fecha, la dirección de la página y el número de página en cada hoja del PDF → Guardar.

2. Comprima el proyecto **sin** las carpetas `node_modules` ni archivos `.env` (los instala/crea quien reciba el proyecto). En PowerShell, desde la carpeta padre:

```
Get-ChildItem .\proyecto-inventario -Recurse -File |  
Where-Object { $_.FullName -notmatch '\\node_modules\\|\\dist\\|\\.git\\|.env$' } |  
Compress-Archive -DestinationPath .\SGI_Libreria_El_Saber.zip -Update
```

3. Verifique que el ZIP incluya: `cliente/`, `servidor/`, `base_datos/sgi_libreria_completo.sql`, `manuales/` (con los PDF) y `README.md`.

Entrega final: ZIP del proyecto + Manual de Usuario.pdf + Manual de Instalación.pdf + Manual Técnico.pdf + sgi_libreria_completo.sql.

Fin del Manual de Instalación. SGI Librería El Saber · Proyecto de grado ADSO — Ficha 3118323 · SENA Regional Tolima · 2026.